

LAPORAN PROJEK UAS
PEMROGRAMAN BERORIENTASI OBYEK

Dosen Pengampu:

ALLIN JUNIKHAH,M.T.

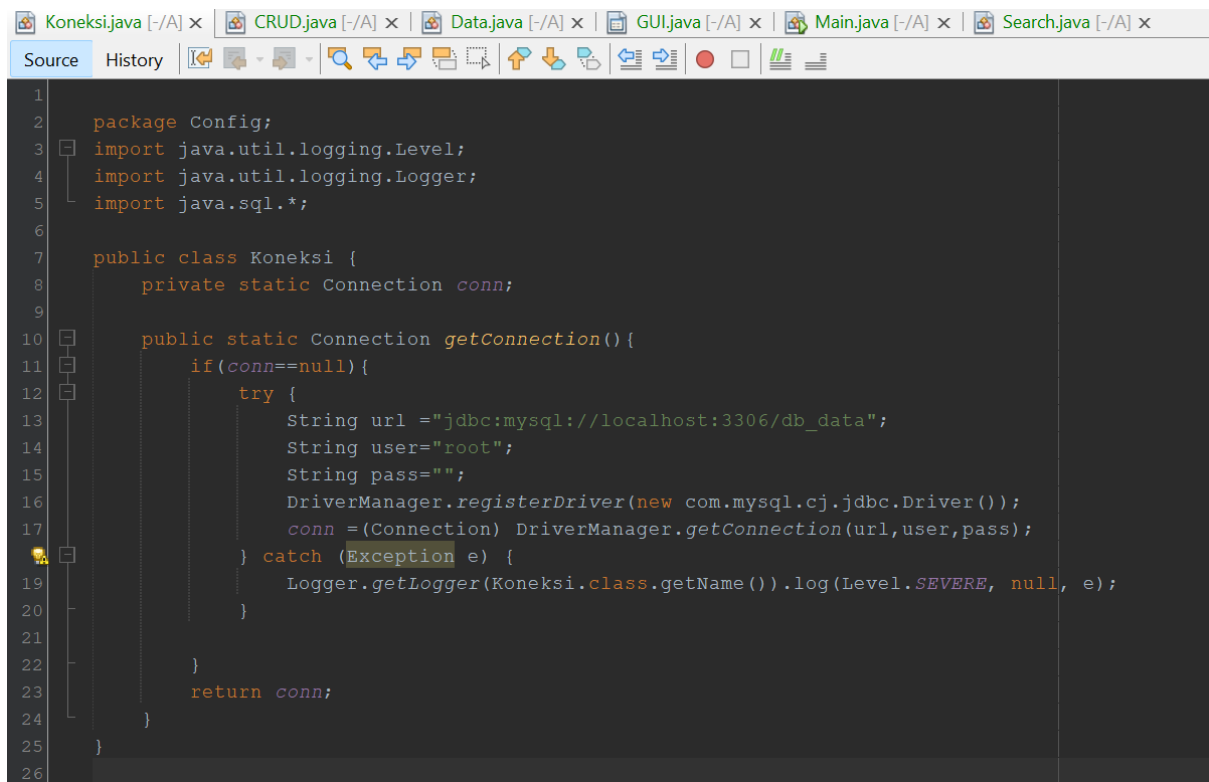


Disusun Oleh:

Rahmat Enomoto (230605110010)

PROGRAM STUDI
TEKNIK INFORMATIKA
FAKULTAS SAINS DAN TEKNOLOGI
UNIVERSITAS ISLAM NEGERI MAULANA MAILK IBRAHIM MALANG
2024

Class Koneksi



```
1
2 package Config;
3 import java.util.logging.Level;
4 import java.util.logging.Logger;
5 import java.sql.*;
6
7 public class Koneksi {
8     private static Connection conn;
9
10    public static Connection getConnection() {
11        if (conn == null) {
12            try {
13                String url = "jdbc:mysql://localhost:3306/db_data";
14                String user = "root";
15                String pass = "";
16                DriverManager.registerDriver(new com.mysql.cj.jdbc.Driver());
17                conn = (Connection) DriverManager.getConnection(url, user, pass);
18            } catch (Exception e) {
19                Logger.getLogger(Koneksi.class.getName()).log(Level.SEVERE, null, e);
20            }
21        }
22        return conn;
23    }
24 }
25
26
```

Class Koneksi berfungsi untuk mengelola koneksi ke database;

Kita dapat koneksi ke database mysql dengan method getConnection();

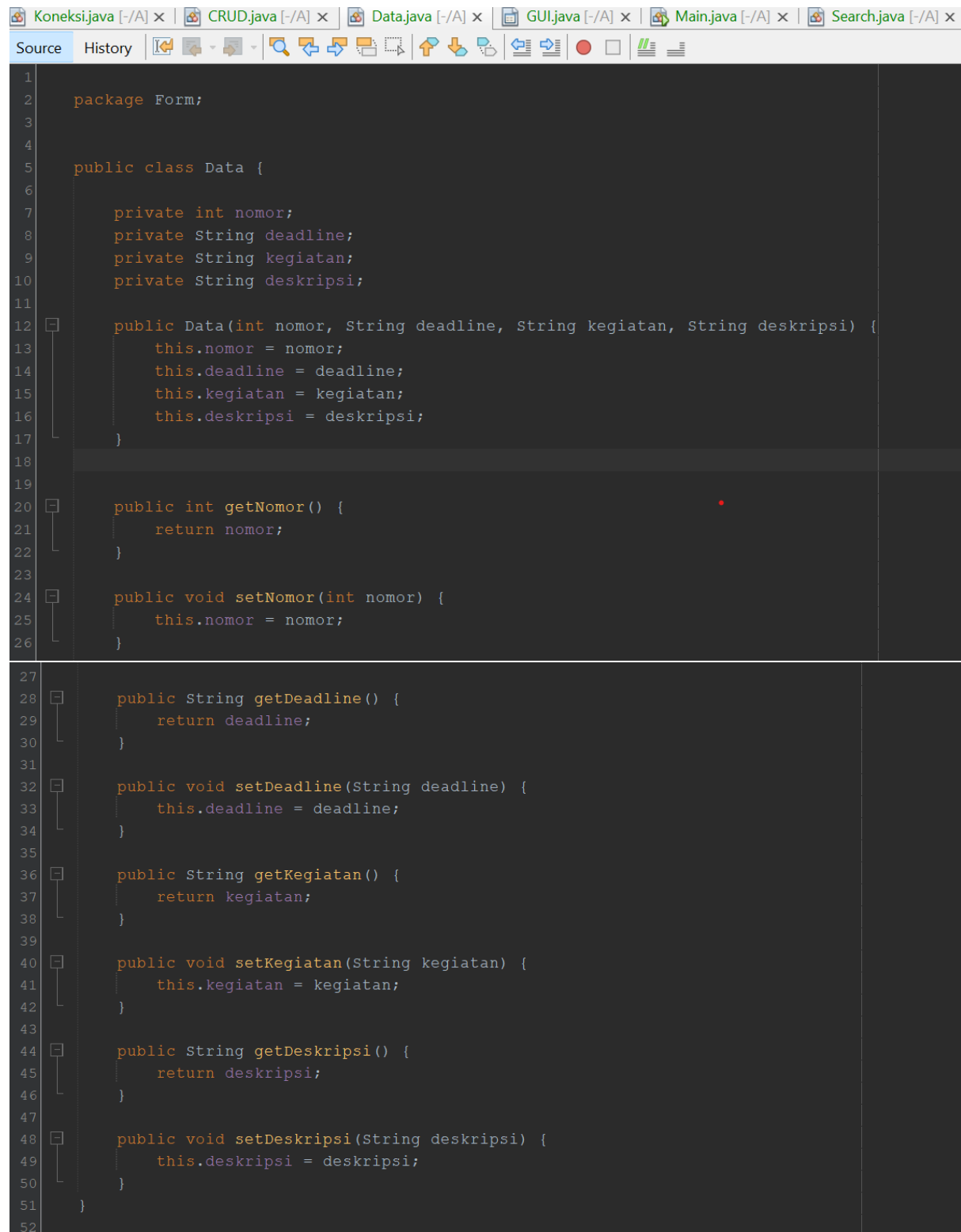
Konsep OOP yang digunakan

- Enkapsulasi
Variable conn bersifat private sehingga hanya dapat di akses melalui metode getConnection

```
private static Connection conn;

public static Connection getConnection() {
```

Class Data



```
1 package Form;
2
3
4
5 public class Data {
6
7     private int nomor;
8     private String deadline;
9     private String kegiatan;
10    private String deskripsi;
11
12    public Data(int nomor, String deadline, String kegiatan, String deskripsi) {
13        this.nomor = nomor;
14        this.deadline = deadline;
15        this.kegiatan = kegiatan;
16        this.deskripsi = deskripsi;
17    }
18
19
20    public int getNomor() {
21        return nomor;
22    }
23
24    public void setNomor(int nomor) {
25        this.nomor = nomor;
26    }
27
28    public String getDeadline() {
29        return deadline;
30    }
31
32    public void setDeadline(String deadline) {
33        this.deadline = deadline;
34    }
35
36    public String getKegiatan() {
37        return kegiatan;
38    }
39
40    public void setKegiatan(String kegiatan) {
41        this.kegiatan = kegiatan;
42    }
43
44    public String getDeskripsi() {
45        return deskripsi;
46    }
47
48    public void setDeskripsi(String deskripsi) {
49        this.deskripsi = deskripsi;
50    }
51 }
52
```

Class Data ini digunakan untuk blueprint data data yang akan di gunakan,

Konsep oop yang di gunakan pada Class ini adalah

- **Konstruktor**

Konstruktor Data memungkinkan pembuatan instance baru dari kelas dengan nilai-nilai awal yang ditentukan. Ini adalah contoh penerapan konstruktor untuk menginisialisasi objek dengan nilai tertentu pada saat pembuatannya.

```
public Data(int nomor, String deadline, String kegiatan, String deskripsi) {  
    this.nomor = nomor;  
    this.deadline = deadline;  
    this.kegiatan = kegiatan;  
    this.deskripsi = deskripsi;  
}
```

- **Enkapsulasi**

Variabel-variabel nomor, deadline, kegiatan, dan deskripsi bersifat privat, yang berarti hanya dapat diakses secara langsung oleh metode-metode dalam kelas Data saja.

```
private int nomor;  
private String deadline;  
private String kegiatan;  
private String deskripsi;
```

Metode getter dan setter menyediakan cara untuk mengakses dan memodifikasi variabel-variabel ini dari luar kelas dengan cara yang terkendali, menjaga integritas data dan memberikan fleksibilitas dalam pengelolaan data.

```
public int getNomor() {  
    return nomor;  
}  
  
public void setNomor(int nomor) {  
    this.nomor = nomor;  
}  
  
public String getDeadline() {  
    return deadline;  
}  
  
public void setDeadline(String deadline) {  
    this.deadline = deadline;  
}  
  
public String getKegiatan() {  
    return kegiatan;  
}  
  
public void setKegiatan(String kegiatan) {  
    this.kegiatan = kegiatan;  
}  
  
public String getDeskripsi() {  
    return deskripsi;  
}
```

Class CRUD

```
Koneksi.java [-/A] x  CRUD.java [-/A] x  Data.java [-/A] x  GUI.java [-/A] x  Main.java [-/A] x  Search.java [-/A] x
Source History
1 package Form;
2
3 import java.sql.Connection;
4 import java.sql.PreparedStatement;
5 import java.sql.ResultSet;
6 import java.util.ArrayList;
7 import java.util.List;
8 import java.util.logging.Level;
9 import java.util.logging.Logger;
10
11 public class CRUD {
12     private Connection conn;
13
14     public CRUD(Connection conn) {
15         this.conn = conn;
16     }
17
18     CRUD() {
19         throw new UnsupportedOperationException("Not supported yet.");
20     }
21
22     public List<Data> getAllData() {
23         List<Data> dataList = new ArrayList<>();
24         try {
25             String sql = "SELECT * FROM mahasiswa ORDER BY nomor ASC";
26             PreparedStatement st = conn.prepareStatement(sql);
27             ResultSet rs = st.executeQuery();
28
29             while (rs.next()) {
30                 int nomor = rs.getInt("nomor");
31                 String deadline = rs.getString("deadline");
32                 String kegiatan = rs.getString("kegiatan");
33                 String deskripsi = rs.getString("deskripsi");
34                 Data data = new Data(nomor, deadline, kegiatan, deskripsi);
35                 dataList.add(data);
36             }
37
38             rs.close();
39             st.close();
40         } catch (Exception e) {
41             Logger.getLogger(CRUD.class.getName()).log(Level.SEVERE, null, e);
42         }
43         return dataList;
44     }
45
46     public boolean addData(Data data) {
47         try {
48             String sql = "INSERT INTO mahasiswa (nomor, deadline, kegiatan, deskripsi) VALUES (?, ?, ?, ?)";
49             PreparedStatement st = conn.prepareStatement(sql);
50             st.setInt(1, data.getNomor());
51             st.setString(2, data.getDeadline());
52             st.setString(3, data.getKegiatan());
53             st.setString(4, data.getDeskripsi());
54
55             int rowInserted = st.executeUpdate();
56
57             st.close();
58             return rowInserted > 0;
59         } catch (Exception e) {
60             Logger.getLogger(CRUD.class.getName()).log(Level.SEVERE, null, e);
61             return false;
62         }
63     }
64
65     public boolean updateData(Data data) {
66         try {
67             String sql = "UPDATE mahasiswa SET deadline = ?, kegiatan = ?, deskripsi = ? WHERE nomor = ?";
68             PreparedStatement st = conn.prepareStatement(sql);
69             st.setString(1, data.getDeadline());
70             st.setString(2, data.getKegiatan());
71             st.setString(3, data.getDeskripsi());
72             st.setInt(4, data.getNomor());
73
74             int rowUpdated = st.executeUpdate();
75             st.close();
76             return rowUpdated > 0;
77         } catch (Exception e) {
78             Logger.getLogger(CRUD.class.getName()).log(Level.SEVERE, null, e);
79             return false;
80         }
81     }
82 }
```

```

82 public boolean deleteData(int nomor) {
83     try {
84         String sql = "DELETE FROM mahasiswa WHERE nomor = ?";
85         PreparedStatement st = conn.prepareStatement(sql);
86         st.setInt(1, nomor);
87
88         int rowDeleted = st.executeUpdate();
89         st.close();
90         return rowDeleted > 0;
91     } catch (Exception e) {
92         Logger.getLogger(CRUD.class.getName()).log(Level.SEVERE, null, e);
93         return false;
94     }
95 }
96 }
97

```

Class CRUD berfungsi sebagai class yang mengelola operasi Create, Read, Update, dan Delete (CRUD) pada tabel mahasiswa dalam database.

Class ini menggunakan objek Connection untuk berinteraksi dengan database dan melakukan operasi SQL untuk menambah, mengambil, memperbarui, dan menghapus data dalam tabel mahasiswa.

Mendapatkan Semua Data (getAllData)

- Metode getAllData dalam kelas CRUD melakukan query SELECT untuk mengambil semua baris dari tabel mahasiswa.
- Untuk setiap baris dalam hasil query, metode ini membuat objek Data baru dan mengisinya dengan data dari baris tersebut.
- Objek Data yang dibuat ditambahkan ke dalam List<Data>, yang kemudian dikembalikan oleh metode ini.

```

public List<Data> getAllData() {
    List<Data> dataList = new ArrayList<>();
    try {
        String sql = "SELECT * FROM mahasiswa ORDER BY nomor ASC";
        PreparedStatement st = conn.prepareStatement(sql);
        ResultSet rs = st.executeQuery();

        while (rs.next()) {
            int nomor = rs.getInt("nomor");
            String deadline = rs.getString("deadline");
            String kegiatan = rs.getString("kegiatan");
            String deskripsi = rs.getString("deskripsi");
            Data data = new Data(nomor, deadline, kegiatan, deskripsi);
            dataList.add(data);
        }

        rs.close();
        st.close();
    } catch (Exception e) {
        Logger.getLogger(CRUD.class.getName()).log(Level.SEVERE, null, e);
    }
    return dataList;
}

```

Menambah Data (addData)

Metode addData menerima objek Data sebagai parameter. Metode ini mengekstrak nilai dari objek Data menggunakan metode getter dan menggunakan nilai-nilai tersebut dalam pernyataan SQL INSERT untuk menambah baris baru ke tabel mahasiswa.

```
public boolean addData(Data data) {
    try {
        String sql = "INSERT INTO mahasiswa (nomor, deadline, kegiatan, deskripsi) VALUES (?, ?, ?, ?)";
        PreparedStatement st = conn.prepareStatement(sql);
        st.setInt(1, data.getNomor());
        st.setString(2, data.getDeadline());
        st.setString(3, data.getKegiatan());
        st.setString(4, data.getDeskripsi());

        int rowInserted = st.executeUpdate();
        st.close();
        return rowInserted > 0;
    } catch (Exception e) {
        Logger.getLogger(CRUD.class.getName()).log(Level.SEVERE, null, e);
        return false;
    }
}
```

Memperbarui Data (updateData)

Metode updateData menerima objek Data sebagai parameter. Metode ini mengekstrak nilai dari objek Data menggunakan metode getter dan menggunakan nilai-nilai tersebut dalam pernyataan SQL UPDATE untuk memperbarui baris yang sesuai dalam tabel mahasiswa.

```
public boolean updateData(Data data) {
    try {
        String sql = "UPDATE mahasiswa SET deadline = ?, kegiatan = ?, deskripsi = ? WHERE nomor = ?";
        PreparedStatement st = conn.prepareStatement(sql);
        st.setString(1, data.getDeadline());
        st.setString(2, data.getKegiatan());
        st.setString(3, data.getDeskripsi());
        st.setInt(4, data.getNomor());

        int rowUpdated = st.executeUpdate();
        st.close();
        return rowUpdated > 0;
    } catch (Exception e) {
        Logger.getLogger(CRUD.class.getName()).log(Level.SEVERE, null, e);
        return false;
    }
}
```

Menghapus Data (deleteData)

Metode deleteData menerima nomor sebagai parameter. Metode ini menggunakan nilai nomor dalam pernyataan SQL DELETE untuk menghapus baris yang sesuai dalam tabel mahasiswa.

```
public boolean deleteData(Data data) {
    try {
        String sql = "DELETE FROM mahasiswa WHERE nomor = ?";
        PreparedStatement st = conn.prepareStatement(sql);
        st.setInt(1, data.getNomor());

        int rowDeleted = st.executeUpdate();
        st.close();
        return rowDeleted > 0;
    } catch (Exception e) {
        Logger.getLogger(CRUD.class.getName()).log(Level.SEVERE, null, e);
        return false;
    }
}
```

Konsep oop yang digunakan dalam Class CRUD:

Enkapsulasi

```
private Connection conn;
```

Konstruktor yang menerima objek Connection sebagai parameter

```
public CRUD(Connection conn) {  
    this.conn = conn;  
}
```


Class Search

```
Koneksi.java [-/A] x | CRUD.java [-/A] x | Data.java [-/A] x | GUI.java [-/A] x | Main.java [-/A] x | Search.java [-/A] x
Source History
1
2 package Form;
3
4 import java.sql.PreparedStatement;
5 import java.sql.ResultSet;
6 import java.sql.Connection;
7 import java.util.ArrayList;
8 import java.util.List;
9 import java.util.logging.Level;
10 import java.util.logging.Logger;
11
12 public class Search {
13     private Connection conn;
14
15     public Search(Connection conn) {
16         this.conn = conn;
17     }
18
19     public List<Data> searchData(String keyword) {
20         List<Data> dataList = new ArrayList<>();
21         try {
22             String sql = "SELECT * FROM mahasiswa WHERE nomor LIKE ? OR deadline LIKE ? OR kegiatan LIKE ? OR deskripsi LIKE ?";
23             PreparedStatement st = conn.prepareStatement(sql);
24             String searchKeyword = "%" + keyword + "%";
25             st.setString(1, searchKeyword);
26             st.setString(2, searchKeyword);
27             st.setString(3, searchKeyword);
28             st.setString(4, searchKeyword);
29
30             ResultSet rs = st.executeQuery();
31             while (rs.next()) {
32                 int nomor = rs.getInt("nomor");
33                 String deadline = rs.getString("deadline");
34                 String kegiatan = rs.getString("kegiatan");
35                 String deskripsi = rs.getString("deskripsi");
36                 Data data = new Data(nomor, deadline, kegiatan, deskripsi);
37                 dataList.add(data);
38             }
39
40             rs.close();
41             st.close();
42         } catch (Exception e) {
43             Logger.getLogger(CRUD.class.getName()).log(Level.SEVERE, null, e);
44         }
45         return dataList;
46     }
47 }
48
49
```

Class Search adalah sebuah class yang bertugas untuk melakukan pencarian data dalam tabel mahasiswa berdasarkan suatu kata kunci (keyword).

Query SQL untuk melakukan pencarian data berdasarkan nomor, deadline, kegiatan, atau deskripsi yang mengandung keyword.

```
String sql = "SELECT * FROM mahasiswa WHERE nomor LIKE ? OR deadline LIKE ? OR kegiatan LIKE ? OR deskripsi LIKE ?";
```

Konsep oop yang digunakan dalam class search yaitu enkapsulasi dan konstruktor

```
private Connection conn;

public Search(Connection conn) {
    this.conn = conn;
}
```

Class GUI

```
package Form;
import Config.Koneksi;
import java.sql.Connection;
import java.util.List;
import javax.swing.JOptionPane;
import javax.swing.table.DefaultTableModel;

public class GUI extends javax.swing.JFrame {

    private final Connection conn;
    private final Search search;
    private final CRUD crud;

    public GUI() {
        initComponents();
        conn = Koneksi.getConnection();
        crud = new CRUD(conn);
        search = new Search(conn);
        getData();
        nonActiveButton();
        activeButton();
    }

    private void getData() {
        DefaultTableModel model = (DefaultTableModel) tbl_data.getModel();
        model.setRowCount(0);

        // Menggunakan metode getAllData dari kelas CRUD
        List<Data> dataList = crud.getAllData();

        for (Data data : dataList) {
            Object[] rowData = {data.getNomor(), data.getDeadline(), data.getKegiatan(), data.getDeskripsi()};
            model.addRow(rowData);
        }

        @SuppressWarnings("unchecked")
        Generated Code

        private void t_deadlineActionPerformed(java.awt.event.ActionEvent evt) {
            // TODO add your handling code here:
        }

        private void t_cariActionPerformed(java.awt.event.ActionEvent evt) {
            // TODO add your handling code here:
        }

        private void btn_updateActionPerformed(java.awt.event.ActionEvent evt) {

            int selectedRow = tbl_data.getSelectedRow();
            if (selectedRow == -1) {
                JOptionPane.showMessageDialog(this, "Pilih baris yang ingin diperbarui");
                return;
            }

            // Menggunakan metode getAllData dari kelas CRUD
            List<Data> dataList = crud.getAllData();

            for (Data data : dataList) {
                Object[] rowData = {data.getNomor(), data.getDeadline(), data.getKegiatan(), data.getDeskripsi()};
                model.addRow(rowData);
            }

            @SuppressWarnings("unchecked")
            Generated Code

            private void t_deadlineActionPerformed(java.awt.event.ActionEvent evt) {
                // TODO add your handling code here:
            }

            private void t_cariActionPerformed(java.awt.event.ActionEvent evt) {
                // TODO add your handling code here:
            }

            private void btn_updateActionPerformed(java.awt.event.ActionEvent evt) {

                int selectedRow = tbl_data.getSelectedRow();
                if (selectedRow == -1) {
                    JOptionPane.showMessageDialog(this, "Pilih baris yang ingin diperbarui");
                    return;
                }
            }
        }
    }
}
```

```

        nonActiveButton();
    }

    private void btn_tambahActionPerformed(java.awt.event.ActionEvent evt) {
        int nomor = Integer.parseInt(t_nomor.getText());
        String deadline = t_deadline.getText();
        String kegiatan = t_kegiatan.getText();
        String deskripsi = t_deskripsi.getText();

        // Validasi
        if (deadline.isEmpty() || kegiatan.isEmpty() || deskripsi.isEmpty()) {
            JOptionPane.showMessageDialog(this, "Semua kolom harus diisi!", "Validasi", JOptionPane.ERROR_MESSAGE);
            return;
        }

        // Menambahkan data menggunakan metode addData dari kelas CRUD
        Data data = new Data(nomor, deadline, kegiatan, deskripsi);
        if (crud.addData(data)) {
            JOptionPane.showMessageDialog(this, "Tugas bertambah, segera dikerjakan!");
            resetForm();
            getData();
        } else {
            JOptionPane.showMessageDialog(this, "Gagal menambahkan data!", "Error", JOptionPane.ERROR_MESSAGE);
        }
    }

```

```

    private void tbl_dataMouseClicked(java.awt.event.MouseEvent evt) {

        int selectedRow=tbl_data.getSelectedRow();
        if(selectedRow != -1){
            int nomor =Integer.parseInt(tbl_data.getValueAt(selectedRow,0).toString());
            String deadline=tbl_data.getValueAt(selectedRow,1).toString();
            String kegiatan=tbl_data.getValueAt(selectedRow,2).toString();
            String deskripsi=tbl_data.getValueAt(selectedRow,3).toString();

            t_nomor.setText(String.valueOf(nomor));
            t_deadline.setText(deadline);
            t_kegiatan.setText(kegiatan);
            t_deskripsi.setText(deskripsi);
        }

        btn_tambah.setEnabled(false);
        btn_update.setEnabled(true);
        btn_hapus.setEnabled(true);
    }

```

```

    private void t_deskripsiKeyReleased(java.awt.event.KeyEvent evt) {
        // TODO add your handling code here:
    }

```

```

    private void btn_hapusActionPerformed(java.awt.event.ActionEvent evt) {
        int selectedRow = tbl_data.getSelectedRow();

```

```

        if (selectedRow == -1) {
            JOptionPane.showMessageDialog(this, "Pilih baris yang ingin dihapus");
            return;
        }

        int confirm = JOptionPane.showConfirmDialog(this, "Apakah Anda yakin ingin menghapus data?", "Konfirmasi", JOptionPane.YES_NO_OPTION);
        if (confirm == JOptionPane.YES_OPTION) {
            // Mengambil nomor dari baris yang dipilih
            int nomor = (int) tbl_data.getValueAt(selectedRow, 0);

            // Menghapus data menggunakan metode deleteData dari kelas CRUD
            if (crud.deleteData(nomor)) {
                JOptionPane.showMessageDialog(this, "Data berhasil dihapus");
                resetForm();
                getData();
            } else {
                JOptionPane.showMessageDialog(this, "Gagal menghapus data!", "Error", JOptionPane.ERROR_MESSAGE);
            }
        }
    }

    private void t_kegiatanActionPerformed(java.awt.event.ActionEvent evt) {
        // TODO add your handling code here:
    }

    private void t_nomorActionPerformed(java.awt.event.ActionEvent evt) {
        // TODO add your handling code here:
    }

```

```

        if (selectedRow == -1) {
            JOptionPane.showMessageDialog(this, "Pilih baris yang ingin dihapus");
            return;
        }

        int confirm = JOptionPane.showConfirmDialog(this, "Apakah Anda yakin ingin menghapus data?", "Konfirmasi", JOptionPane.YES_NO_OPTION);
        if (confirm == JOptionPane.YES_OPTION) {
            // Mengambil nomor dari baris yang dipilih
            int nomor = (int) tbl_data.getValueAt(selectedRow, 0);

            // Menghapus data menggunakan metode deleteData dari kelas CRUD
            if (crud.deleteData(nomor)) {
                JOptionPane.showMessageDialog(this, "Data berhasil dihapus");
                resetForm();
                getData();
            } else {
                JOptionPane.showMessageDialog(this, "Gagal menghapus data!", "Error", JOptionPane.ERROR_MESSAGE);
            }
        }
    }

    private void t_kegiatanActionPerformed(java.awt.event.ActionEvent evt) {
        // TODO add your handling code here:
    }

    private void t_nomorActionPerformed(java.awt.event.ActionEvent evt) {
        // TODO add your handling code here:
    }
}

```

```

    private void t_cariKeyTyped(java.awt.event.KeyEvent evt) {
        DefaultTableModel model = (DefaultTableModel) tbl_data.getModel();
        model.setRowCount(0);

        // Mendapatkan kata kunci pencarian
        String keyword = t_cari.getText();

        // Mencari data menggunakan metode searchData dari kelas CRUD
        List<Data> dataList = search.searchData(keyword);

        // Mengisi tabel dengan hasil pencarian
        for (Data data : dataList) {
            Object[] rowData = {data.getNomor(), data.getDeadline(), data.getKegiatan(), data.getDeskripsi()};
            model.addRow(rowData);
        }
    }

    private void t_cariMouseClicked(java.awt.event.MouseEvent evt) {
        t_cari.setText("");
    }

    private void btn_tambahActionPerformed(java.awt.event.ActionEvent evt) {
        // TODO add your handling code here:
    }
}

```

```

// Variables Declaration - do not modify
private javax.swing.JButton btn_batal;
private javax.swing.JButton btn_hapus;
private javax.swing.JButton btn_tambah;
private javax.swing.JButton btn_tambah1;
private javax.swing.JButton btn_update;
private javax.swing.JLabel dmfsd;
private javax.swing.JLabel dmfsd1;
private javax.swing.JLabel dmfsd2;
private javax.swing.JLabel dmfsd3;
private javax.swing.JLabel dmfsd4;
private javax.swing.JLabel dmfsd5;
private javax.swing.JLabel dmfsd6;
private javax.swing.JLabel dmfsd7;
private javax.swing.JLabel dmfsd8;
private javax.swing.JLabel jLabel1;
private javax.swing.JLabel jLabel2;
private javax.swing.JLabel jLabel3;
private javax.swing.JLabel jLabel4;
private javax.swing.JLabel jLabel5;
private javax.swing.JLabel jLabel6;
private javax.swing.JLabel jLabel7;
private javax.swing.JPanel jPanel1;
private javax.swing.JScrollPane jScrollPane1;
private javax.swing.JTextField t_cari;
private javax.swing.JTextField t_deadline;
private javax.swing.JTextField t_deskripsi;

```

```

private javax.swing.JTextField t_cari;
private javax.swing.JTextField t_deadline;
private javax.swing.JTextField t_deskripsi;
private javax.swing.JTextField t_kegiatan;
private javax.swing.JTextField t_nomor;
private javax.swing.JTable tbl_data;
// End of variables declaration

private void resetForm() {
    t_nomor.setText("");
    t_deadline.setText("");
    t_kegiatan.setText("");
    t_deskripsi.setText("");
}
private void nonActiveButton() {
    btn_update.setEnabled(false);
    btn_batal.setEnabled(false);
    btn_hapus.setEnabled(false);
}
private void activeButton() {
    btn_tambah.setEnabled(true);
    btn_batal.setEnabled(true);
}
private String t_nomor.getText() {
    throw new UnsupportedOperationException("Not supported yet."); // Generated from nbfs://nbhost/SystemFileSystem/Templates
}
}

```

Class GUI adlah sebuah implementasi dari (Graphical User Interface) dalam Java menggunakan Java Swing untuk membuat sebuah aplikasi To-Do List sederhana.

Deklarasi variabel dan objek

- conn: Objek koneksi ke database.
- search: Objek dari kelas Search untuk melakukan pencarian data.
- crud: Objek dari kelas CRUD untuk melakukan operasi Create, Read, Update, dan Delete terhadap data.

Ini juga memakai konsep oop yaitu enkapsulasi

```

private final Connection conn;
private final Search search;
private final CRUD crud;

```

Konstruktor GUI

- Konstruktor ini dipanggil saat objek GUI dibuat. Langkah-langkah yang dilakukan antara lain: Menginisialisasi komponen GUI dengan initComponents().
- Mendapatkan koneksi ke database menggunakan Koneksi.getConnection().
- Menginisialisasi objek crud dan search dengan menggunakan koneksi.
- Memuat data awal dari database ke dalam tabel menggunakan getData().
- Mengatur tombol-tombol untuk aktif/nonaktif menggunakan nonActiveButton() dan activeButton().

```

public GUI() {
    initComponents();
    conn = Koneksi.getConnection();
    crud = new CRUD(conn);
    search = new Search(conn);
    getData();
    nonActiveButton();
    activeButton();
}

```

Method `getData()`;

- Metode ini mengambil semua data dari database menggunakan objek crud (dari kelas CRUD).
- Data yang diambil kemudian dimasukkan ke dalam model tabel (`DefaultTableModel`) untuk ditampilkan di GUI.

```
private void getData() {
    DefaultTableModel model = (DefaultTableModel) tbl_data.getModel();
    model.setRowCount(0);

    // Menggunakan metode getAllData dari kelas CRUD
    List<Data> dataList = crud.getAllData();

    for (Data data : dataList) {
        Object[] rowData = {data.getNomor(), data.getDeadline(), data.getKegiatan(), data.getDeskripsi()};
        model.addRow(rowData);
    }
}
```

Event dan Aksi pengguna

Terdapat berbagai method event handling seperti `btn_tambahActionPerformed`, `btn_updateActionPerformed`, `btn_hapusActionPerformed`, `tbl_dataMouseClicked`, dan lainnya yang menangani interaksi pengguna terhadap elemen GUI seperti tombol dan tabel. Ini menggunakan koneksi ke class CRUD untuk melakukan operasi seperti `deleteData`, `addData`, `updateData`, `getAllData`

Contoh:

```
private void btn_hapusActionPerformed(java.awt.event.ActionEvent evt) {
    int selectedRow = tbl_data.getSelectedRow();
    if (selectedRow == -1) {
        JOptionPane.showMessageDialog(this, "Pilih baris yang ingin dihapus");
        return;
    }

    int confirm = JOptionPane.showConfirmDialog(this, "Apakah Anda yakin ingin menghapus data?", "Konfirmasi", JOptionPane.YES_NO_OPTION);
    if (confirm == JOptionPane.YES_OPTION) {
        // Mengambil nomor dari baris yang dipilih
        int nomor = (int) tbl_data.getValueAt(selectedRow, 0);

        // Menghapus data menggunakan metode deleteData dari kelas CRUD
        if (crud.deleteData(nomor)) {
            JOptionPane.showMessageDialog(this, "Data berhasil dihapus");
            resetForm();
            getData();
        } else {
            JOptionPane.showMessageDialog(this, "Gagal menghapus data!", "Error", JOptionPane.ERROR_MESSAGE);
        }
    }
}
```

Dan Mencari data menggunakan metode `saarchData` dari class `Search`

```
private void t_cariKeyTyped(java.awt.event.KeyEvent evt) {
    DefaultTableModel model = (DefaultTableModel) tbl_data.getModel();
    model.setRowCount(0);

    // Mendapatkan kata kunci pencarian
    String keyword = t_cari.getText();

    // Mencari data menggunakan metode searchData dari kelas CRUD
    List<Data> dataList = search.searchData(keyword);

    // Mengisi tabel dengan hasil pencarian
    for (Data data : dataList) {
        Object[] rowData = {data.getNomor(), data.getDeadline(), data.getKegiatan(), data.getDeskripsi()};
        model.addRow(rowData);
    }
}
```

resetForm()

Metode ini digunakan untuk mengatur ulang (reset) semua input di dalam form ke nilai awal (kosong). Hal ini berguna setelah data telah ditambahkan atau dihapus, sehingga form siap untuk entri baru.

```
private void resetForm() {  
    t_nomor.setText("");  
    t_deadline.setText("");  
    t_kegiatan.setText("");  
    t_deskripsi.setText("");  
}
```

nonActiveButton()

Metode ini digunakan untuk menonaktifkan tombol-tombol tertentu dalam antarmuka pengguna. Ini berguna untuk mencegah pengguna melakukan tindakan yang tidak diinginkan ketika kondisi tertentu tidak terpenuhi.

```
private void nonActiveButton() {  
    btn_update.setEnabled(false);  
    btn_batal.setEnabled(false);  
    btn_hapus.setEnabled(false);  
}
```

activeButton()

Metode ini digunakan untuk mengaktifkan tombol-tombol tertentu dalam antarmuka pengguna. Ini memungkinkan pengguna untuk melakukan tindakan yang diinginkan ketika kondisi tertentu terpenuhi.

```
private void activeButton() {  
    btn_tambah.setEnabled(true);  
    btn_batal.setEnabled(true);  
}
```

Class Main

```
package Form;

public class Main {
    public static void main(String[] args) {
        java.awt.EventQueue.invokeLater(new Runnable() {
            public void run() {
                new GUI().setVisible(true);
            }
        });
    }
}
```

Class Main sebagai tempat untuk menjalankan GUI

new GUI().

setVisible(true) Di dalam metode run, sebuah instans dari kelas GUI dibuat. Kemudian, metode setVisible(true) dipanggil untuk membuat jendela GUI menjadi terlihat.

```
new GUI().setVisible(true);
```


Laragon Full 6.0 220916 php-8.1.10-Win32-vs16-x64 [TS] 10.0.85.252



Menu

Apache httpd-2.4.54-win64-VS16 started80Reload

MySQL mysql-8.0.30-winx64 started3306

Stop

Web

Database

Terminal

Root

Server: localhost » Database: db_data » Tabel: mahasiswa

Jelajahi

Struktur

SQL

Cari

Tambahkan

Ekspor

Impor

⚠ Current selection does not contain a unique column. Grid edit, checkbox, Edit, Copy and Delete features

✓ Menampilkan baris 0 - 3 (total 4, Pencarian dilakukan dalam 0,0003 detik.)

SELECT * FROM `mahasiswa`

☐ Profil [Edit kotak] [Ubah] [Jelaskan SQL] [Buat kode PHP] [Segarkan]

☐ Tampilkan semua

Jumlah baris: 25

Saring baris: Cari di tabel ini

Extra options

nomor	deadline	kegiatan	deskripsi
1	22 juni 2024	uas pkn	membuat essay x
3	14	makan	tidur
2	7 juli	uas mtk	peluang dadu
4	9 mei	uas b	uas

Tampilan

To Do LIST

Nomor :

Deadline :

Kegiatan :

Deskripsi :

tambah

update

batal

hapus

pencarian :

Nomor	DeadLine	Kegiatan	Deskripsi
1	22 juni 2024	uas pkn	membuat essay x
2	7 juli	uas mtk	peluang dadu
3	14	makan	tidur
4	9 mei	uas b	uas

To Do LIST

Nomor :

Deadline :

Kegiatan :

Deskripsi :

tambah

update

batal

hapus

pencarian :

3

Nomor	DeadLine	Kegiatan	Deskripsi
3	14	makan	tidur